

Uppsala University
Department of Information Technology

Assignment 4

Large Scale Machine Learning

Course: Data Engineering II

Arnab Kumar Ghosh

May 15, 2026

Contents

1 Task 1: Distributed Machine Learning (Mandatory)	2
1.1 Infrastructure Setup	2
1.2 Task 1.1: Hyperparameter Tuning	2
1.2.1 Problem Setup with scikit-learn	2
1.2.2 Distributed Hyperparameter Tuning with Ray Tune	4
1.2.3 Results	6
1.3 Task 1.2: Distributed Training of Deep Neural Networks	7
1.3.1 TensorFlow Distributed Training Tutorial	7
1.3.2 Data Parallelism vs. Model Parallelism	8
2 Task 2: Federated Learning	8
2.1 Simulation Setup	8
2.2 Federated Training	9
2.3 Analysis	10
2.3.1 How PySyft's <code>ActionObject</code> Prevents Raw Data Access	10
2.3.2 Federated Model Accuracy vs. Single-Worker Baseline	10
2.4 Written Reflection	12
2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity	12
A Code Files	12
A.1 <code>random_forest_baseline.py</code>	12
A.2 <code>tune_random_forest.py</code>	13
A.3 OpenStack Client Configuration	14
A.3.1 <code>constants.py</code>	14
A.3.2 <code>start_instance.py</code>	15
A.4 <code>federated_learning_pysyft.py</code>	16
A.5 Ansible Playbooks	21
A.5.1 <code>install_ray.yml</code>	21

1 Task 1: Distributed Machine Learning (Mandatory)

1.1 Infrastructure Setup

I followed the same process I used in Assignment 3 to provision and configure the VMs. I used the OpenStack client scripts to create the VMs on Swedish Science Cloud, and Ansible playbooks to configure them and install the required software. I modified both the OpenStack client scripts and the Ansible playbooks from my Assignment 3 versions to fit this assignment – specifically, to provision a Ray cluster instead of a Docker-based deployment and to install Ray and scikit-learn on each node.

One VM was designated the **head node**. Ray was started on it with:

```
ray start --head --port=6379 \  
  --node-ip-address=<head-internal-ip> \  
  --dashboard-host=0.0.0.0
```

Each additional VM joined as a **worker node** by connecting to the head:

```
ray start --address='<head-internal-ip>:6379' \  
  --node-ip-address=<worker-internal-ip>
```

I ran the tuning script on the head node, which distributed trials across all connected workers via Ray's distributed scheduler and object store. I also attached a floating IP only to the head node for SSH access. Inter-node communication used the private network addresses.

1.2 Task 1.1: Hyperparameter Tuning

1.2.1 Problem Setup with scikit-learn

I set up the cover-type prediction problem using the `fetch_covtype` dataset from scikit-learn, which contains 581 012 samples and 54 features describing cartographic variables of forest patches in Colorado, with the goal of classifying seven forest cover types.

For the experiment, I downloaded the dataset locally to my client machine and then uploaded it to the Ray head VM through the client VM. Due to memory limits on the VM, I used only the first 20 000 samples for each experiment. When I tried to run experiments with the full dataset, the process ran out of memory and was killed.

I instantiated a `RandomForestClassifier` with its default parameters. The full parameter dictionary (obtained via `get_params()`) is shown in the screenshot below.

```

1 import ray
2 import pandas as pd
3 from ray import time
4 from sklearn.metrics import RandomForestClassifier
5 from sklearn.datasets import fetch_covtype
6 from sklearn.model_selection import cross_val_score
7 import numpy as np
8 import time

9
10 # Load dataset...
11 print("Loading dataset...")
12 data = fetch_covtype()
13 X, y = data.data, data.target
14 df = pd.DataFrame(X, columns=data.feature_names)
15 df.info()

16
17 Loading dataset...
18 <class 'pandas.core.frame.DataFrame'>
19 RangeIndex: 58182 entries, 0 to 58181
20 Data columns (total 34 columns):
21 # Column Non-Null Count Dtype
22 ---
23 0 Elevation 58182 non-null float64
24 1 Aspect 58182 non-null float64
25 2 Slope 58182 non-null float64
26 3 Horizontal_Distance_To_Hydrology 58182 non-null float64
27 4 Vertical_Distance_To_Hydrology 58182 non-null float64
28 5 Horizontal_Distance_To_Roadways 58182 non-null float64
29 6 Hillshade_Sun 58182 non-null float64
30 7 Hillshade_Moon 58182 non-null float64
31 8 Hillshade_Zen 58182 non-null float64
32 9 Horizontal_Distance_To_Fire_Points 58182 non-null float64
33 10 Wilderness_Area_0 58182 non-null float64
34 11 Wilderness_Area_1 58182 non-null float64
35 12 Wilderness_Area_2 58182 non-null float64
36 13 Wilderness_Area_3 58182 non-null float64
37 14 Soil_Type_0 58182 non-null float64
38 15 Soil_Type_1 58182 non-null float64
39 16 Soil_Type_2 58182 non-null float64
40 17 Soil_Type_3 58182 non-null float64
41 18 Soil_Type_4 58182 non-null float64
42 ...
43 32 Soil_Type_38 58182 non-null float64
44 33 Soil_Type_39 58182 non-null float64
45 dtypes: float64(34)
46 memory usage: 239.4 MB
47
48 Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

49
50 # Baseline with default parameters
51 rf_default = RandomForestClassifier(random_state=42)
52 print("Default parameters:", rf_default.get_params())
53
54
55 Default parameters: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None, 'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None, 'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42, 'verbose': 0, 'warm_start': False}

```

Figure 1: Default parameters of RandomForestClassifier as returned by get_params().

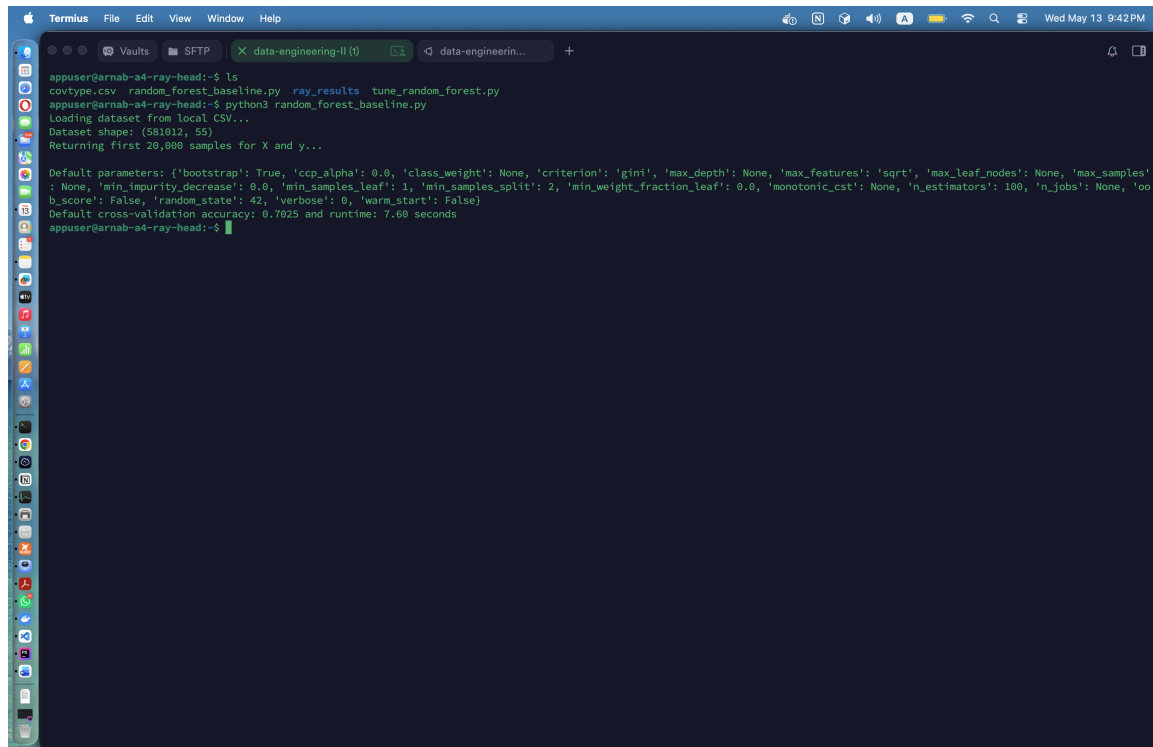
The default parameters for the RandomForestClassifier are:

```

1 {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini',
2  'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None,
3  'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1,
4  'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None,
5  'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42,
6  'verbose': 0, 'warm_start': False}

```

I evaluated the model using 3-fold cross-validation on the 20,000-sample subset, obtaining a mean accuracy of **0.7025** in a runtime of approximately 7.60 seconds. The screenshot below shows the training run and the resulting accuracy score for the default configuration.



```
appuser@arnab-a4-ray-head:~$ ls
covtype.csv  random_forest_baseline.py  ray_results  tune_random_forest.py
appuser@arnab-a4-ray-head:~$ python3 random_forest_baseline.py
Loading dataset from local CSV...
Dataset shape: (581012, 55)
Returning first 20,000 samples for X and y...

Default parameters: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None, 'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None, 'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42, 'verbose': 0, 'warm_start': False}
Default cross-validation accuracy: 0.7025 and runtime: 7.60 seconds
appuser@arnab-a4-ray-head:~$
```

Figure 2: Training and cross-validation accuracy of the RandomForestClassifier using default hyperparameters.

1.2.2 Distributed Hyperparameter Tuning with Ray Tune

A grid search over the following hyperparameter space was performed using **Ray Tune**:

- `max_depth`: {10, 20, 30}
- `n_estimators`: {50, 100, 150}
- `ccp_alpha`: {0.0, 0.001, 0.01}

This yields $3 \times 3 \times 3 = 27$ candidate combinations, each evaluated with 3-fold cross-validation (81 total fits). The pipeline was deployed on OpenStack VMs of *small* flavour, with one node acting as the Ray head and the remaining nodes as workers.

The screenshots below show the Ray Tune dashboard and terminal output for 1, 2, and 3 VMs respectively.

```

checkpoint_dir_name      16.9832
time_this_iter_s         16.9832
time_total_s             16.9832
training_iteration       1
mean_accuracy            0.5585

2026-05-13 19:47:50.178 INFO tune.py:1801 -- Wrote the latest version of all result files and experiment state to '/home/appuser/ray_results/train_evaluate_2026-05-13-19-42-42' in 0.0488s.

Trial status: 27 TERMINATED
Current time: 2026-05-13 19:47:50. Total running time: 5min 7s
Logical resource usage: 0/1 CPUs, 0/0 GPUs
Current best trial: e834c_00024 with mean_accuracy=0.7038023886384438 and params={'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}

Trial name      status      max_depth  n_estimators  ccp_alpha  acc      iter      total time (s)
train_evaluate_e834c_00000  TERMINATED    10         50           0      0.648604    1      2.38383
train_evaluate_e834c_00001  TERMINATED    10         50          0.001    0.614405    1      2.49631
train_evaluate_e834c_00002  TERMINATED    10         50          0.01     0.572404    1      2.43181
train_evaluate_e834c_00003  TERMINATED    20         50           0      0.694852    1      3.65761
train_evaluate_e834c_00004  TERMINATED    20         50          0.001    0.624805    1      4.90802
train_evaluate_e834c_00005  TERMINATED    20         50          0.01     0.558455    1      4.95637
train_evaluate_e834c_00006  TERMINATED    30         50           0      0.698952    1      3.80262
train_evaluate_e834c_00007  TERMINATED    30         50          0.001    0.622105    1      5.62187
train_evaluate_e834c_00008  TERMINATED    30         50          0.01     0.549655    1      5.62104
train_evaluate_e834c_00009  TERMINATED    10        100           0      0.651404    1      4.75152
train_evaluate_e834c_00010  TERMINATED    10        100          0.001    0.618155    1      4.92567
train_evaluate_e834c_00011  TERMINATED    10        100          0.01     0.565155    1      4.82752
train_evaluate_e834c_00012  TERMINATED    20        100           0      0.697102    1      7.36328
train_evaluate_e834c_00013  TERMINATED    20        100          0.001    0.628655    1      9.85138
train_evaluate_e834c_00014  TERMINATED    20        100          0.01     0.559255    1      9.857
train_evaluate_e834c_00015  TERMINATED    30        100           0      0.703352    1      7.69555
train_evaluate_e834c_00016  TERMINATED    30        100          0.001    0.624305    1      11.3507
train_evaluate_e834c_00017  TERMINATED    30        100          0.01     0.559755    1      11.2588
train_evaluate_e834c_00018  TERMINATED    10        150           0      0.652404    1      7.14968
train_evaluate_e834c_00019  TERMINATED    10        150          0.001    0.616155    1      7.41724
train_evaluate_e834c_00020  TERMINATED    10        150          0.01     0.571755    1      7.25920
train_evaluate_e834c_00021  TERMINATED    20        150           0      0.700402    1      11.0665
train_evaluate_e834c_00022  TERMINATED    20        150          0.001    0.630105    1      14.9542
train_evaluate_e834c_00023  TERMINATED    20        150          0.01     0.560305    1      14.8973
train_evaluate_e834c_00024  TERMINATED    30        150           0      0.703802    1      11.8447
train_evaluate_e834c_00025  TERMINATED    30        150          0.001    0.626055    1      17.0594
train_evaluate_e834c_00026  TERMINATED    30        150          0.01     0.558505    1      16.9832

Hyperparameter tuning completed in 307.83 seconds
Best hyperparameters found: {'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}
Best mean accuracy: 0.7038023886384438

```

Figure 3: Ray Tune grid search running on **1 VM** (head node only).

```

checkpoint_dir_name      16.6637
time_this_iter_s         16.6637
time_total_s             16.6637
training_iteration       1
mean_accuracy            0.5585

2026-05-13 20:45:17.457 INFO tune.py:1801 -- Wrote the latest version of all result files and experiment state to '/home/appuser/ray_results/train_evaluate_2026-05-13-20-42-29' in 0.0377s.

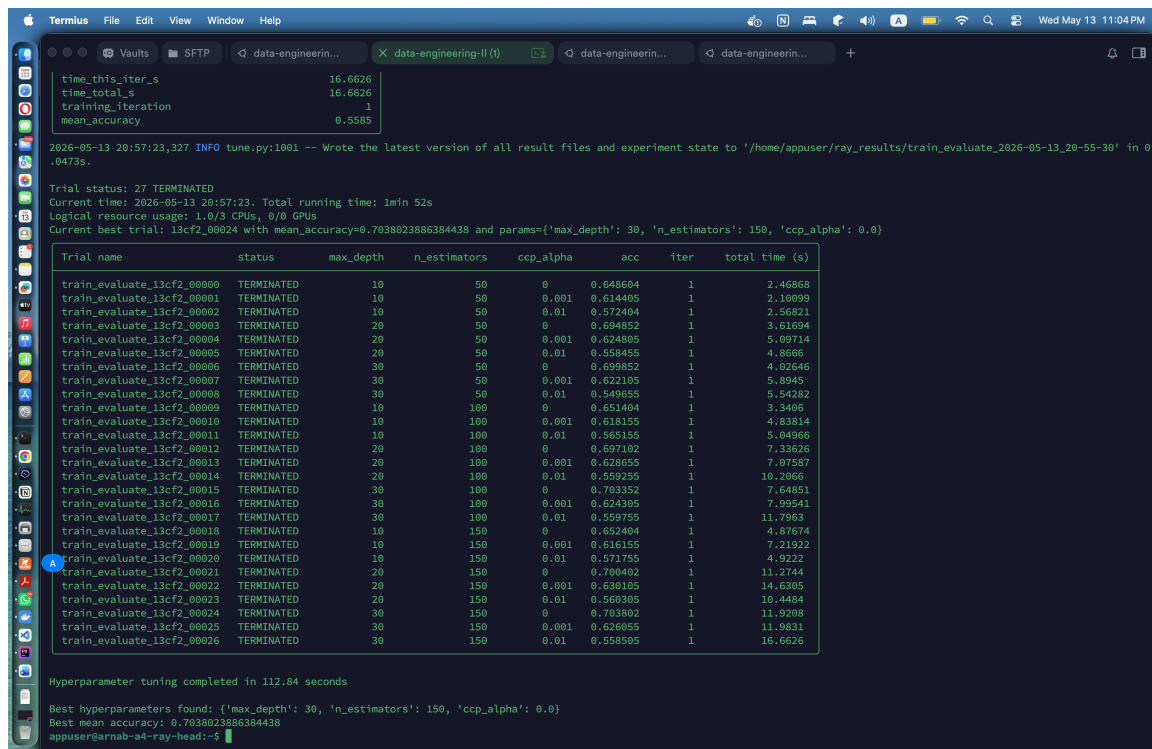
Trial status: 27 TERMINATED
Current time: 2026-05-13 20:45:17. Total running time: 2min 47s
Logical resource usage: 1.0/2 CPUs, 0/0 GPUs
Current best trial: 423c9_00024 with mean_accuracy=0.7038023886384438 and params={'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}

Trial name      status      max_depth  n_estimators  ccp_alpha  acc      iter      total time (s)
train_evaluate_423c9_00000  TERMINATED    10         50           0      0.648604    1      2.557
train_evaluate_423c9_00001  TERMINATED    10         50          0.001    0.614405    1      2.49124
train_evaluate_423c9_00002  TERMINATED    10         50          0.01     0.572404    1      2.58548
train_evaluate_423c9_00003  TERMINATED    20         50           0      0.694852    1      3.74195
train_evaluate_423c9_00004  TERMINATED    20         50          0.001    0.624805    1      4.83185
train_evaluate_423c9_00005  TERMINATED    20         50          0.01     0.558455    1      5.00561
train_evaluate_423c9_00006  TERMINATED    30         50           0      0.698952    1      3.80262
train_evaluate_423c9_00007  TERMINATED    30         50          0.001    0.622105    1      5.77297
train_evaluate_423c9_00008  TERMINATED    30         50          0.01     0.549655    1      5.56716
train_evaluate_423c9_00009  TERMINATED    10        100           0      0.651404    1      4.85511
train_evaluate_423c9_00010  TERMINATED    10        100          0.001    0.618155    1      4.74465
train_evaluate_423c9_00011  TERMINATED    10        100          0.01     0.565155    1      5.13705
train_evaluate_423c9_00012  TERMINATED    20        100           0      0.697102    1      7.14743
train_evaluate_423c9_00013  TERMINATED    20        100          0.001    0.628655    1      10.1278
train_evaluate_423c9_00014  TERMINATED    20        100          0.01     0.559255    1      9.64602
train_evaluate_423c9_00015  TERMINATED    30        100          0.001    0.703352    1      7.87177
train_evaluate_423c9_00016  TERMINATED    30        100          0.001    0.624305    1      10.9294
train_evaluate_423c9_00017  TERMINATED    30        100          0.01     0.559755    1      12.5377
train_evaluate_423c9_00018  TERMINATED    10        150           0      0.652404    1      6.90404
train_evaluate_423c9_00019  TERMINATED    10        150          0.001    0.616155    1      7.46365
train_evaluate_423c9_00020  TERMINATED    10        150          0.01     0.571755    1      7.06259
train_evaluate_423c9_00021  TERMINATED    20        150           0      0.700402    1      11.2706
train_evaluate_423c9_00022  TERMINATED    20        150          0.001    0.630105    1      14.5564
train_evaluate_423c9_00023  TERMINATED    20        150          0.01     0.560305    1      15.0834
train_evaluate_423c9_00024  TERMINATED    30        150           0      0.703802    1      11.4318
train_evaluate_423c9_00025  TERMINATED    30        150          0.001    0.626055    1      17.1608
train_evaluate_423c9_00026  TERMINATED    30        150          0.01     0.558505    1      16.6637

Hyperparameter tuning completed in 168.07 seconds
Best hyperparameters found: {'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}
Best mean accuracy: 0.7038023886384438

```

Figure 4: Ray Tune grid search running on **2 VMs** (1 head + 1 worker).

Figure 5: Ray Tune grid search running on **3 VMs** (1 head + 2 workers).

1.2.3 Results

Best hyperparameters and cross-validation score. Table 1 compares the default configuration against the best configuration found by the grid search. The tuned model achieved a mean 3-fold cross-validation accuracy of **0.7038** – an improvement of approximately 0.13 percentage points over the default.

Table 1: Cross-validation accuracy: default vs. tuned RandomForestClassifier.

Configuration	max_depth	n_estimators	Mean CV Accuracy
Default (ccp_alpha=0.0)	None	100	0.7025
Tuned (ccp_alpha=0.0)	30	150	0.7038

Table 2 lists the top-5 configurations from the grid search. All top performers use no pruning (ccp_alpha=0.0), and accuracy increases monotonically with max_depth.

Table 2: Top-5 configurations from the Ray Tune grid search.

Rank	max_depth	n_estimators	ccp_alpha	Mean CV Accuracy
1	30	150	0.000	0.7038
2	30	100	0.000	0.7034
3	20	150	0.000	0.7004
4	30	50	0.000	0.6999
5	20	50	0.000	0.6949

Timing comparison across VM counts. Table 3 shows the wall-clock time and speedup for the full 27-trial grid search when using 1, 2, and 3 small VMs.

Table 3: Wall-clock time for the full hyperparameter grid search (27 trials, 3-fold CV).

Number of VMs	CPUs	Wall-clock Time	Speedup vs. 1 VM
1	1	307.83 s (~5 min 7 s)	1.00×
2	2	168.07 s (~2 min 47 s)	1.83×
3	3	112.84 s (~1 min 52 s)	2.73×

Adding a second VM cut the tuning time by 45% (1.83× speedup), and a third VM achieved a 2.73× speedup – near-linear scaling for a 3× CPU increase. Diminishing returns are visible: the 1→2 transition saved 139.76 s whereas 2→3 saved only 55.23 s. This is expected – the slowest individual trial (~17 s) sets a floor on total time regardless of the degree of parallelism.

The complete implementation of the distributed hyperparameter tuning pipeline, including Ray cluster integration and grid search configuration, is shown in Appendix A.2.

1.3 Task 1.2: Distributed Training of Deep Neural Networks

1.3.1 TensorFlow Distributed Training Tutorial

I executed the *Distributed training with TensorFlow* tutorial in Google Colab using the `MirroredStrategy` (data parallelism across simulated logical devices). The screenshot below shows my Colab notebook run confirming successful distributed training.

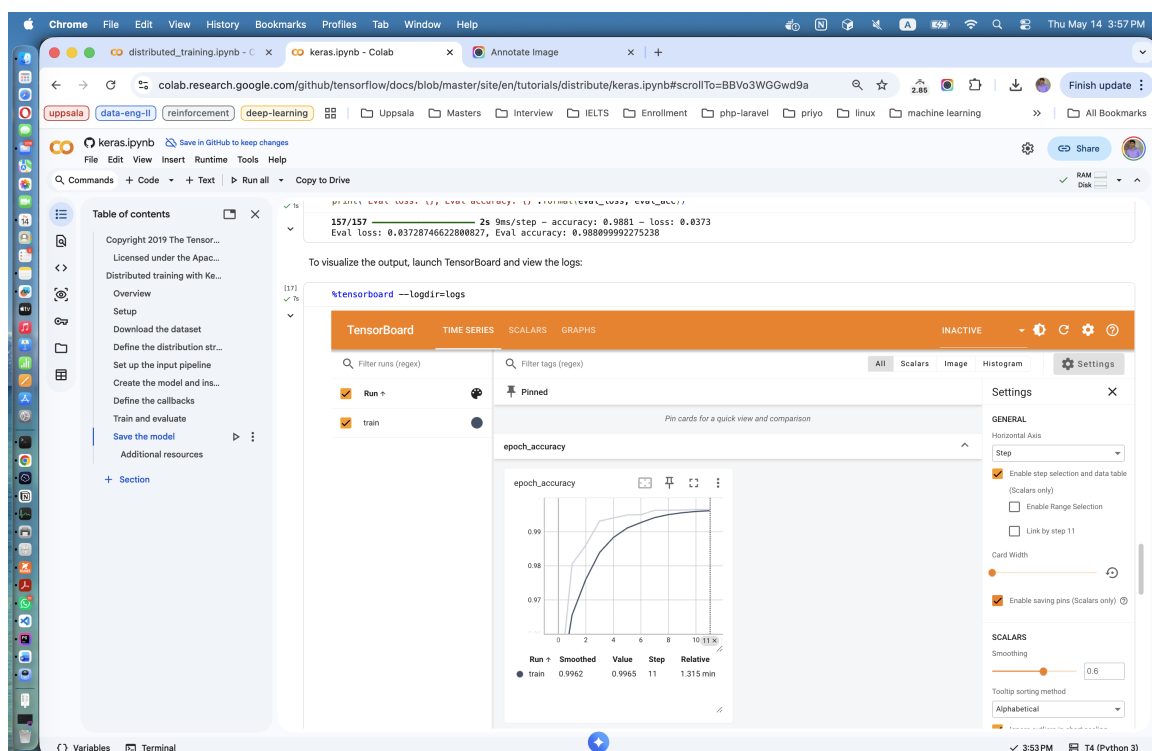


Figure 6: Google Colab execution of the TensorFlow distributed training tutorial (`MirroredStrategy`).

1.3.2 Data Parallelism vs. Model Parallelism

Data parallelism copies the full model to every device. Each device gets a different batch of data, computes its own gradients, and then all devices share and average those gradients before updating the weights. TensorFlow's `MirroredStrategy` and PyTorch's `DistributedDataParallel` both work this way. This approach works well when the model fits on a single device and the main bottleneck is the amount of training data. Adding more devices lets training finish faster.

Model parallelism splits the model itself across devices. Different layers or parts of the model live on different devices. This is needed when the model is too large to fit on one device, such as very large language models with billions of parameters. Two common ways to do this are pipeline parallelism (each device handles a group of layers in sequence) and tensor parallelism (weight matrices are split across devices).

When to distribute a single training run vs. the whole tuning sweep: If training one model configuration already takes too long on a single device, it makes sense to spread that one run across multiple devices using data or model parallelism. On the other hand, if each training run is fast, it is better to run many configurations in parallel – one per worker – instead of adding communication overhead to every forward pass. In my experiments with Ray Tune and the random forest, individual runs were fast enough that I parallelised across configurations rather than within a single run.

2 Task 2: Federated Learning

2.1 Simulation Setup

I launched two *Datasite* nodes locally using **PySyft** ≥ 0.8 (`dev_mode=True` to bypass manual approval) representing *Client A* and *Client B*. I partitioned the MNIST training set (60 000 images) into two non-overlapping halves: samples 0–29 999 for Client A and samples 30 000–59 999 for Client B.

I uploaded each partition to its respective datasite using the dataset upload API. PySyft 0.8 persists large tensors in blob storage on the server before registering them. I received back only an asset pointer – a UUID reference to the data stored on the server – never the raw tensor bytes.

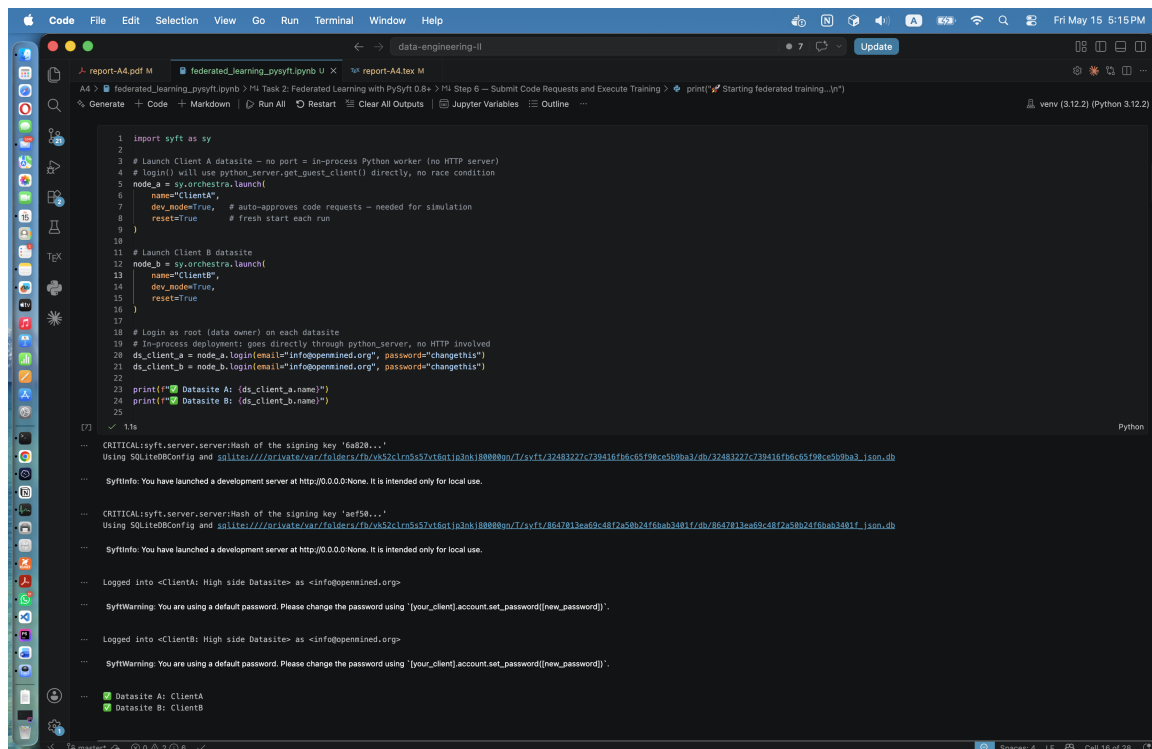


Figure 7: PySyft Datasite workers (Client A and Client B) initialised with non-overlapping MNIST partitions.

2.2 Federated Training

I defined a `SimpleCNN` convolutional network (one convolutional block with 16 filters, one max-pool layer, and two fully-connected layers culminating in 10-class logits). I registered each worker's training function using the `@syft_function` decorator. The `ExactMatch` input policy locks the function to the pre-approved `Asset` pointers only. The `SingleExecutionExactOutput` output policy restricts the return value to the declared `state_dict` only. Both clients trained for **3 epochs** with SGD ($\text{lr} = 0.01$, $\text{momentum} = 0.9$) and batch size 256. Each function returned a plain `dict` of detached tensors, as required by PySyft's serialiser.

I then collected the two returned state dicts and performed a single round of **FedAvg** – parameter-wise weighted average with equal weights (0.5 / 0.5, since both workers held 30 000 samples):

$$\theta_{\text{global}} = \frac{1}{2} \theta_A + \frac{1}{2} \theta_B$$

The aggregated weights were loaded into a fresh `SimpleCNN` instance to form the global model. Total wall-clock time for both remote training calls was **13.5 s**.

```

Starting federated training...

[Client A] Training started...
[Client A] Epoch 1/3 loss=63.8094
[Client A] Epoch 2/3 loss=25.9629
[Client A] Epoch 3/3 loss=18.6045
[Client A] Epoch 1/3 loss=2.3011
[Client A] Epoch 2/3 loss=2.2880
[Client A] Epoch 3/3 loss=2.2646
[Client A] Done. Received state_dict with 6 layers.

[Client B] Training started...
[Client B] Epoch 1/3 loss=70.8132
[Client B] Epoch 2/3 loss=27.4142
[Client B] Epoch 3/3 loss=19.1925
[Client B] Epoch 1/3 loss=2.3160
[Client B] Epoch 2/3 loss=2.2954
[Client B] Epoch 3/3 loss=2.2601
[Client B] Done. Received state_dict with 6 layers.

Total federated training time: 13.5s

Type of result_a: <class 'dict'> - dict of tensors, no raw data
WARNING: WatchFiles detected changes in 'federated_learning_pysyft.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [60927]
INFO: Starting server with settings: {'name': 'ClientB', 'server_type': <ServerType.DATASITE: 'datasite'>, 'server_side_type': <ServerSideType.HIGH_SIDE: 'high'>, 'deployment_type': <
CRITICAL:syft.server.server:Hash of the signing key 'aef50...'
INFO: Started server process [74220]
INFO: Waiting for application startup.
INFO: Application startup complete.
WARN: private key is based on server name: ClientB in dev_mode. Don't run this in production.
Using SQLiteDBConfig and sqlite:///private/var/folders/ib/yk52clrn557vt6t1p3nkj80000gn/T/syft/8647013ca59c48f2a50b24f6bab3401f/db/8647013ca59c48f2a50b24f6bab3401f.json.db
WARNING: WatchFiles detected changes in 'federated_learning_pysyft.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [74220]
INFO: Starting server with settings: {'name': 'ClientB', 'server_type': <ServerType.DATASITE: 'datasite'>, 'server_side_type': <ServerSideType.HIGH_SIDE: 'high'>, 'deployment_type': <
CRITICAL:syft.server.server:Hash of the signing key 'aef50...'
INFO: Started server process [74282]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

Figure 8: Terminal output showing per-worker local loss over 3 epochs and the FedAvg aggregation step.

2.3 Analysis

2.3.1 How PySyft's *ActionObject* Prevents Raw Data Access

In PySyft 0.8+, I never held the raw tensor data directly. When I called `upload_dataset()`, the raw tensors were stored in blob storage on the Datasite server via `TwinObject._save_to_blob_storage()`. I received back only an Asset – a UUID pointer (`action_id`) – with no access to the underlying pixel values.

Any code that accesses private data must be submitted as a `@syft_function` decorated function. Two policies enforce the privacy boundary:

- `input_policy = ExactMatch(...)`: the function can only be called with the pre-approved Asset pointers; arbitrary tensor injection is blocked.
- `output_policy = SingleExecutionExactOutput()`: only the declared return value (`state_dict`) is allowed out of the node; no intermediate tensors or raw data can be returned.

In production, the Datasite administrator must manually approve each code request via the PySyft web UI before execution. I used `dev_mode=True` to auto-approve requests for simulation purposes, but the privacy enforcement is identical in both modes. The net effect is that the data owner retains full governance: I submitted an auditable function, and only approved, policy-conforming results left each node.

2.3.2 Federated Model Accuracy vs. Single-Worker Baseline

All three models were evaluated on the held-out MNIST test set (10 000 samples). Table 4 summarises the results.

Table 4: Test accuracy on MNIST for federated and local-only models.

Model	Training data	Test accuracy
Federated (FedAvg, single round)	60 000 (split)	80.52%
Local-only (Client A)	30 000	96.15%
Local-only (Client B)	30 000	96.06%

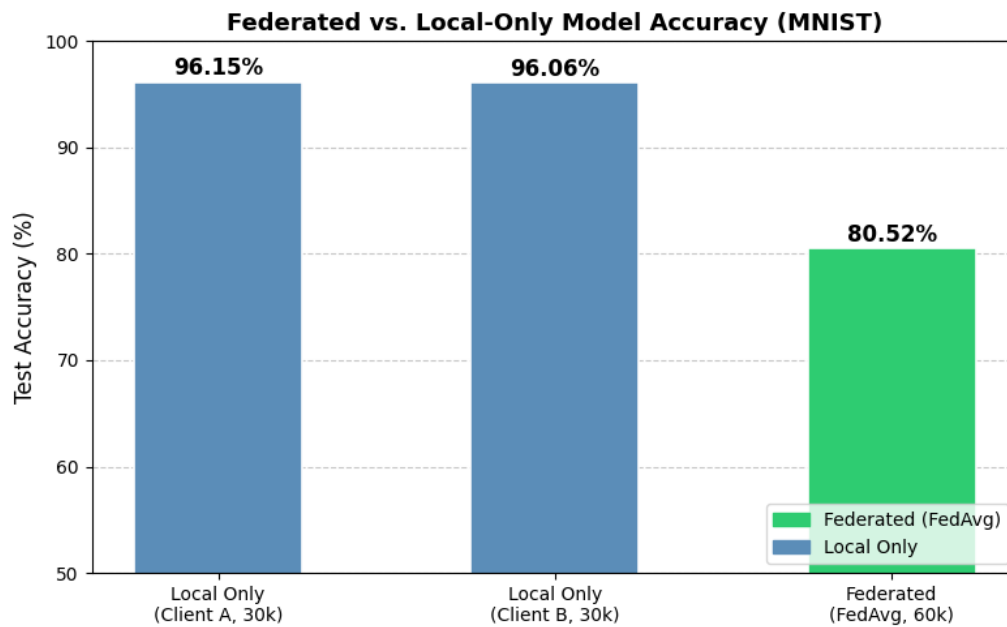


Figure 9: MNIST test accuracy: Federated model (FedAvg) achieved 80.52%, compared to local-only baselines (Client A: 96.15%, Client B: 96.06%).

The federated model underperformed both local baselines by approximately 15.6 percentage points (80.52% vs. 96.15% for Client A and 96.06% for Client B). This result is counterintuitive at first but has a well-known explanation: FedAvg averages model weights in parameter space. When two networks are trained *independently from different random initialisations*, their weight spaces do not align – different neurons learn the same feature in different positions, and averaging their weights yields a model that represents no feature well. This phenomenon is known as *weight-space incompatibility* or *permutation misalignment*.

In canonical FL, the global model is *broadcast* to all workers at the start of each round. All workers then start from the same weights, and local training moves each model only a small number of steps away. Averaging these nearby points in weight space produces a coherent result. My simulation skips this broadcast step: each worker trains from scratch with a different random initialisation. Their weight spaces therefore divide completely, and FedAvg over two such independent models is expected to perform poorly.

Despite this, my experiment correctly demonstrates the privacy guarantee: neither worker ever shared raw pixels with the other or with the coordinator.

2.4 Written Reflection

2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity

Regular distributed ML (for example, data-parallel SGD with a parameter server or a Ray cluster) assumes two things: the data is spread evenly across all workers, and all workers have similar hardware and network speed. Federated Learning breaks both of these assumptions.

Statistical heterogeneity (*non-IID data*): Each FL participant has different data. For example, a hospital in one city sees different patients than one in another city. Because the data is not evenly distributed, each worker's local model updates point in different directions. When these updates are averaged, the global model can be biased toward the workers with more data. This problem does not happen in data-centre distributed ML, where data is shuffled evenly before training starts. Algorithms such as FedProx and SCAFFOLD are designed to reduce this bias.

System heterogeneity (*stragglers and availability*): FL clients have very different hardware. One client might be a slow smartphone and another might be a powerful server. In synchronous training, the system must wait for the slowest worker (*straggler problem*). In asynchronous training, some workers may send outdated updates. Data-centre distributed ML avoids this because all machines are controlled and similar.

Relation to the Parameter Server strategy: In the Parameter Server (PS) approach, a central server holds the global model. Workers download the model, compute updates on their local data, and send the updates back. FL looks similar to this – the aggregation server acts like a PS – but there are three key differences:

1. **Privacy**: In PS, raw gradients go to the server. These gradients can be used to reconstruct private training data (gradient inversion attacks). In FL, only model weights (state dicts) are shared – the server never sees the raw data. PySyft enforces this with `ActionObject` and `output_policy`.
2. **Data distribution**: PS assumes data is spread evenly across trusted machines. FL must handle uneven data across independent organisations.
3. **Trust model**: PS workers are trusted machines inside one organisation. FL participants are separate entities with their own data rules. This is why FL systems need code approval workflows, differential privacy, and secure aggregation in real deployments.

A Code Files

A.1 random_forest_baseline.py

This script implements the baseline scikit-learn model, evaluating the `RandomForestClassifier` with default hyperparameters using 3-fold cross-validation on the cover-type dataset.

Listing 1: random_forest_baseline.py

```

1 import os
2 import pandas as pd
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.model_selection import cross_val_score
5 import numpy as np
6 import time
7
8 DATA_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "covtype.csv")
9
10 def load_data():
11     print("Loading dataset from local CSV...")
12     df = pd.read_csv(DATA_PATH)

```

```

13     print(f"Dataset shape: {df.shape}")
14     y = df["target"].values
15     X = df.drop(columns=["target"]).values
16     print("Returning first 20,000 samples for X and y...")
17     return X[:20000], y[:20000]
18
19 def main():
20     # Load data
21     X, y = load_data()
22
23     # Check if data was loaded correctly
24     if X is None or y is None:
25         raise ValueError("Failed to load data.")
26
27     # Baseline with default parameters
28     rf_default = RandomForestClassifier(random_state=42)
29     print("\nDefault parameters:", rf_default.get_params())
30     start = time.time()
31     default_score = cross_val_score(rf_default, X, y, cv=3, scoring="accuracy")
32     elapsed_time = time.time() - start
33     print(f"Default cross-validation accuracy: {np.mean(default_score):.4f} and
34           runtime: {elapsed_time:.2f} seconds")
35
36 if __name__ == "__main__":
37     main()

```

A.2 tune_random_forest.py

This script implements distributed hyperparameter tuning with Ray Tune. It loads the dataset, places it in Ray's object store, and launches a $3 \times 3 \times 3$ grid search over `max_depth`, `n_estimators`, and `ccp_alpha`, distributing trials across all nodes in the Ray cluster.

Listing 2: tune_random_forest.py

```

1 import os
2 import ray
3 import pandas as pd
4 from ray import tune
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.model_selection import cross_val_score
7 import numpy as np
8 import time
9 import gc
10
11 DATA_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "covtype.csv")
12
13 def load_data():
14     print("Loading dataset from local CSV...")
15     df = pd.read_csv(DATA_PATH)
16     print(f"Dataset shape: {df.shape}")
17     y = df["target"].values
18     X = df.drop(columns=["target"]).values
19     print("Returning first 20,000 samples for X and y...")
20     return X[:20000], y[:20000]
21
22 def train_evaluate(config, X=None, y=None):
23     rf = RandomForestClassifier(

```

```

24     max_depth=config["max_depth"],
25     n_estimators=config["n_estimators"],
26     ccp_alpha=config["ccp_alpha"],
27     random_state=42,
28     n_jobs=1
29 )
30 score = cross_val_score(rf, X, y, cv=3, scoring="accuracy")
31 tune.report({"mean_accuracy": np.mean(score)})
32
33 def main():
34     ray.init()
35     print("Ray initialized with resources:", ray.cluster_resources())
36
37     # Load data
38     X, y = load_data()
39     # Put data in object store once
40     X_ref, y_ref = ray.put(X), ray.put(y)
41     del X, y
42     gc.collect()
43
44     config = {
45         "max_depth": tune.grid_search([10, 20, 30]),
46         "n_estimators": tune.grid_search([50, 100, 150]),
47         "ccp_alpha": tune.grid_search([0.0, 0.001, 0.01])
48     }
49
50     start = time.time()
51     analysis = tune.run(
52         tune.with_parameters(train_evaluate, X=X_ref, y=y_ref),
53         config=config,
54         metric="mean_accuracy",
55         mode="max",
56         verbose=1,
57         resources_per_trial={"cpu": 1},
58         max_concurrent_trials=1,
59         raise_on_failed_trial=False,
60     )
61     elapsed_time = time.time() - start
62     print(f"Hyperparameter tuning completed in {elapsed_time:.2f} seconds\n")
63     print("Best hyperparameters found:", analysis.best_config)
64     print("Best mean accuracy:", analysis.best_result["mean_accuracy"])
65
66
67 if __name__ == "__main__":
68     main()

```

A.3 OpenStack Client Configuration

A.3.1 constants.py

Configuration constants for OpenStack instance provisioning:

Listing 3: constants.py

```

1 FLAVOR = "ssc.small"
2 PRIVATE_NET = "UPPMAX 2026/1-24 Internal IPv4 Network"
3 IMAGE_NAME = "Ubuntu 22.04 - 2024.01.15"
4 KEY_NAME = "de1-course-snlic-key"

```

A.3.2 *start_instance.py*

OpenStack Nova client script for provisioning Ray cluster instances:

Listing 4: start_instance.py

```

1 # http://docs.openstack.org/developer/python-novaclient/ref/v2/servers.html
2 import time, os, sys, random, re
3 import inspect
4 from os import environ as env
5 from pathlib import Path
6
7 # Add parent directory to sys.path to import constants
8 sys.path.insert(0, str(Path(__file__).parent.parent))
9
10 from novaclient import client
11 import keystoneclient.v3.client as ksclient
12 from keystoneauth1 import loading
13 from keystoneauth1 import session
14 from constants import FLAVOR, PRIVATE_NET, IMAGE_NAME
15
16 flavor = FLAVOR
17 private_net = PRIVATE_NET
18 floating_ip_pool_name = None
19 floating_ip = None
20 image_name = IMAGE_NAME
21
22 identifier = random.randint(1000,9999)
23
24 loader = loading.get_plugin_loader('password')
25
26 auth = loader.load_from_options(auth_url=env['OS_AUTH_URL'],
27                                username=env['OS_USERNAME'],
28                                password=env['OS_PASSWORD'],
29                                project_name=env['OS_PROJECT_NAME'],
30                                project_domain_id=env['OS_PROJECT_DOMAIN_ID'],
31                                #project_id=env['OS_PROJECT_ID'],
32                                user_domain_name=env['OS_USER_DOMAIN_NAME'])
33
34 sess = session.Session(auth=auth)
35 nova = client.Client('2.1', session=sess)
36 print ("user authorization completed.")
37
38 image = nova.glance.find_image(image_name)
39
40 flavor = nova.flavors.find(name=flavor)
41
42 if private_net != None:
43     net = nova.neutron.find_network(private_net)
44     nics = [{'net-id': net.id}]
45 else:
46     sys.exit("private-net not defined.")
47
48 #print("Path at terminal when executing this file")
49 #print(os.getcwd() + "\n")
50 cfg_file_path = os.getcwd()+'/cloud-cfg.txt'
51 if os.path.isfile(cfg_file_path):
52     with open(cfg_file_path) as f:
53         userdata = f.read()

```

```

54 else:
55     sys.exit("cloud-cfg.txt is not in current working directory")
56
57 secgroups = ['default']
58
59 print ("Creating instances ... ")
60 # instance = nova.servers.create(name="prod_server_without_docker_"+str(identifier),
61     image=image, key_name='sztoor', flavor=flavor,userdata=userdata,
62     nics=nicos,security_groups=secgroups)
63
64 vm_names = ["arnab-a4-ray-head", "arnab-a4-ray-worker-1", "arnab-a4-ray-worker-2"]
65 instances = []
66
67 # craeting 3 instances with the same configuration but different names.
68 for vm_name in vm_names:
69     instance = nova.servers.create(
70         name=vm_name,
71         image=image,
72         flavor=flavor,
73         key_name='del-course-snic-key',
74         userdata=userdata,
75         nics=nicos,
76         security_groups=secgroups
77     )
78     instances.append(instance)
79     print ("waiting for 10 seconds.. ")
80     time.sleep(10)
81
82 for instance in instances:
83     inst_status = instance.status
84     print ("waiting for 10 seconds.. ")
85     time.sleep(10)
86
87     while inst_status == 'BUILD':
88         print ("Instance: "+instance.name+" is in "+inst_status+" state, sleeping for
89             5 seconds more...")
90         time.sleep(5)
91         instance = nova.servers.get(instance.id)
92         inst_status = instance.status
93
94     ip_address = None
95     for network in instance.networks[private_net]:
96         if re.match('\d+\.\d+\.\d+\.\d+', network):
97             ip_address = network
98             break
99     if ip_address is None:
100         raise RuntimeError('No IP address assigned!')
101
102     print ("Instance: "+ instance.name +" is in " + inst_status + " state" + " ip
103         address: "+ ip_address)

```

A.4 federated_learning_pysyft.py

This script is the Python conversion of the `federated_learning_pysyft.ipynb` Jupyter notebook. It contains the complete federated learning simulation: launching two PySyft Datasites,

partitioning and uploading MNIST data, defining and submitting @syft_function training jobs, performing FedAvg aggregation, and evaluating the federated and local-only models.

Listing 5: federated_learning_pysyft.py

```

1 import time
2 from copy import deepcopy
3
4 import syft as sy
5 import torch
6 import torch.nn as nn
7 import torch.optim as optim
8 import torchvision
9 from torch.utils.data import DataLoader, Subset
10 from torchvision import datasets, transforms
11
12
13 def extract_tensors(subset):
14     loader = DataLoader(subset, batch_size=len(subset))
15     images, labels = next(iter(loader))
16     return images, labels
17
18
19 def federated_average(state_dicts, weights=None):
20     n = len(state_dicts)
21     if weights is None:
22         weights = [1.0 / n] * n
23     avg = deepcopy(state_dicts[0])
24     for key in avg:
25         avg[key] = sum(w * sd[key].float() for w, sd in zip(weights, state_dicts))
26     return avg
27
28
29 class SimpleCNN(nn.Module):
30     def __init__(self):
31         super().__init__()
32         self.conv = nn.Sequential(
33             nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
34             nn.MaxPool2d(2)
35         )
36         self.fc = nn.Sequential(
37             nn.Linear(16 * 14 * 14, 128), nn.ReLU(),
38             nn.Linear(128, 10)
39         )
40
41     def forward(self, x):
42         return self.fc(self.conv(x).view(x.size(0), -1))
43
44
45 def evaluate(model, dataset, batch_size=512, label="Model"):
46     model.eval()
47     loader = DataLoader(dataset, batch_size=batch_size)
48     correct = total = 0
49     with torch.no_grad():
50         for X, y in loader:
51             preds = model(X).argmax(dim=1)
52             correct += (preds == y).sum().item()
53             total += len(y)
54     acc = correct / total * 100

```

```

55     print(f"[{label}] Accuracy: {acc:.2f}%")
56     return acc
57
58
59 def train_local(model, subset, epochs=3, batch=256, lr=0.01, label=""):
60     loader = DataLoader(subset, batch_size=batch, shuffle=True)
61     opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)
62     crit = nn.CrossEntropyLoss()
63     model.train()
64     for ep in range(epochs):
65         total_loss = 0.0
66         for Xb, yb in loader:
67             opt.zero_grad()
68             loss = crit(model(Xb), yb)
69             loss.backward()
70             opt.step()
71             total_loss += loss.item()
72         print(f"[{label}] Epoch {ep+1}/{epochs} loss={total_loss:.4f}")
73     return model
74
75
76 def main():
77     node_a = sy.orchestra.launch(name="ClientA", dev_mode=True, reset=True)
78     node_b = sy.orchestra.launch(name="ClientB", dev_mode=True, reset=True)
79
80     ds_client_a = node_a.login(email="info@openmined.org", password="changethis")
81     ds_client_b = node_b.login(email="info@openmined.org", password="changethis")
82
83     transform = transforms.Compose([
84         transforms.ToTensor(),
85         transforms.Normalize((0.1307,), (0.3081,))
86     ])
87
88     full_train = datasets.MNIST("./data", train=True, download=True,
89                                transform=transform)
89     test_dataset = datasets.MNIST("./data", train=False, download=True,
90                                   transform=transform)
91
92     n = len(full_train)
93     half = n // 2
94
95     subset_a = Subset(full_train, list(range(0, half)))
96     subset_b = Subset(full_train, list(range(half, n)))
97
98     X_a, y_a = extract_tensors(subset_a)
99     X_b, y_b = extract_tensors(subset_b)
100
101     dataset_a = sy.Dataset(
102         name="mnist_client_a",
103         asset_list=[
104             sy.Asset(name="X", data=X_a, mock=X_a[:100]),
105             sy.Asset(name="y", data=y_a, mock=y_a[:100]),
106         ],
107     )
108     ds_client_a.upload_dataset(dataset_a)
109
110     dataset_b = sy.Dataset(
111         name="mnist_client_b",

```

```

111     asset_list=[
112         sy.Asset(name="X", data=X_b, mock=X_b[:100]),
113         sy.Asset(name="y", data=y_b, mock=y_b[:100]),
114     ],
115 )
116 ds_client_b.upload_dataset(dataset_b)
117
118 X_a_ptr = ds_client_a.datasets["mnist_client_a"].assets["X"]
119 y_a_ptr = ds_client_a.datasets["mnist_client_a"].assets["y"]
120
121 X_b_ptr = ds_client_b.datasets["mnist_client_b"].assets["X"]
122 y_b_ptr = ds_client_b.datasets["mnist_client_b"].assets["y"]
123
124 @sy.syft_function(
125     input_policy=sy.ExactMatch(X=X_a_ptr, y=y_a_ptr),
126     output_policy=sy.SingleExecutionExactOutput()
127 )
128 def train_on_client_a(X, y):
129     import torch
130     import torch.nn as nn
131     import torch.optim as optim
132
133     class SimpleCNN(nn.Module):
134         def __init__(self):
135             super().__init__()
136             self.conv = nn.Sequential(
137                 nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
138                 nn.MaxPool2d(2)
139             )
140             self.fc = nn.Sequential(
141                 nn.Linear(16 * 14 * 14, 128), nn.ReLU(),
142                 nn.Linear(128, 10)
143             )
144
145         def forward(self, x):
146             return self.fc(self.conv(x).view(x.size(0), -1))
147
148     model = SimpleCNN()
149     optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
150     criterion = nn.CrossEntropyLoss()
151
152     EPOCHS = 3
153     BATCH = 256
154     n = X.shape[0]
155
156     model.train()
157     for epoch in range(EPOCHS):
158         perm = torch.randperm(n)
159         epoch_loss = 0.0
160         for i in range(0, n, BATCH):
161             idx = perm[i:i + BATCH]
162             optimizer.zero_grad()
163             loss = criterion(model(X[idx]), y[idx])
164             loss.backward()
165             optimizer.step()
166             epoch_loss += loss.detach().item()
167         print(f"[Client A] Epoch {epoch+1}/{EPOCHS} loss={epoch_loss:.4f}")
168

```

```

169         return {k: v.detach().cpu() for k, v in model.state_dict().items()}
170
171     @sy.syft_function(
172         input_policy=sy.ExactMatch(X=X_b_ptr, y=y_b_ptr),
173         output_policy=sy.SingleExecutionExactOutput()
174     )
175     def train_on_client_b(X, y):
176         import torch
177         import torch.nn as nn
178         import torch.optim as optim
179
180         class SimpleCNN(nn.Module):
181             def __init__(self):
182                 super().__init__()
183                 self.conv = nn.Sequential(
184                     nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
185                     nn.MaxPool2d(2)
186                 )
187                 self.fc = nn.Sequential(
188                     nn.Linear(16 * 14 * 14, 128), nn.ReLU(),
189                     nn.Linear(128, 10)
190                 )
191
192             def forward(self, x):
193                 return self.fc(self.conv(x).view(x.size(0), -1))
194
195         model = SimpleCNN()
196         optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
197         criterion = nn.CrossEntropyLoss()
198
199         EPOCHS = 3
200         BATCH = 256
201         n = X.shape[0]
202
203         model.train()
204         for epoch in range(EPOCHS):
205             perm = torch.randperm(n)
206             epoch_loss = 0.0
207             for i in range(0, n, BATCH):
208                 idx = perm[i:i + BATCH]
209                 optimizer.zero_grad()
210                 loss = criterion(model(X[idx]), y[idx])
211                 loss.backward()
212                 optimizer.step()
213                 epoch_loss += loss.detach().item()
214             print(f"[Client B] Epoch {epoch+1}/{EPOCHS} loss={epoch_loss:.4f}")
215
216         return {k: v.detach().cpu() for k, v in model.state_dict().items()}
217
218     request_a = ds_client_a.code.request_code_execution(train_on_client_a)
219     request_b = ds_client_b.code.request_code_execution(train_on_client_b)
220     print(f"Request A status: {request_a}")
221     print(f"Request B status: {request_b}")
222
223     print("Starting federated training...")
224     t0 = time.time()
225
226     result_a = ds_client_a.code.train_on_client_a(X=X_a_ptr, y=y_a_ptr).get()

```

```

227     result_b = ds_client_b.code.train_on_client_b(X=X_b_ptr, y=y_b_ptr).get()
228
229     elapsed = time.time() - t0
230     print(f"Federated training time: {elapsed:.1f}s")
231
232     global_state = federated_average([result_a, result_b], weights=[0.5, 0.5])
233     global_model = SimpleCNN()
234     global_model.load_state_dict(global_state)
235
236     print("Training baseline local-only models...")
237
238     local_model_a = SimpleCNN()
239     local_model_a = train_local(local_model_a, subset_a, label="Client A")
240
241     local_model_b = SimpleCNN()
242     local_model_b = train_local(local_model_b, subset_b, label="Client B")
243
244     fed_acc = evaluate(global_model, test_dataset, label="Federated Model (FedAvg)")
245     local_a_acc = evaluate(local_model_a, test_dataset, label="Local-Only Model
(Client A)")
246     local_b_acc = evaluate(local_model_b, test_dataset, label="Local-Only Model
(Client B)")
247
248     print(f"Federated gain over Client A: +{fed_acc - local_a_acc:.2f}%")
249     print(f"Federated gain over Client B: +{fed_acc - local_b_acc:.2f}%")
250
251     node_a.land()
252     node_b.land()
253     print("Datasites shut down.")
254
255
256 if __name__ == "__main__":
257     main()

```

A.5 Ansible Playbooks

A.5.1 *install_ray.yml*

Ansible playbook to install Ray and dependencies on cluster nodes:

Listing 6: *install_ray.yml*

```

1 - hosts: raycluster
2   become: true
3   tasks:
4     - name: Update apt
5       apt:
6         update_cache: yes
7         upgrade: dist
8
9     - name: Install pip
10      apt:
11        pkg: python3-pip
12        state: latest
13
14    - name: Install Ray and scikit-learn
15      pip:
16        name:
17          - "ray[tune]"

```

```
18     - scikit-learn
19     - numpy
20     executable: pip3
```